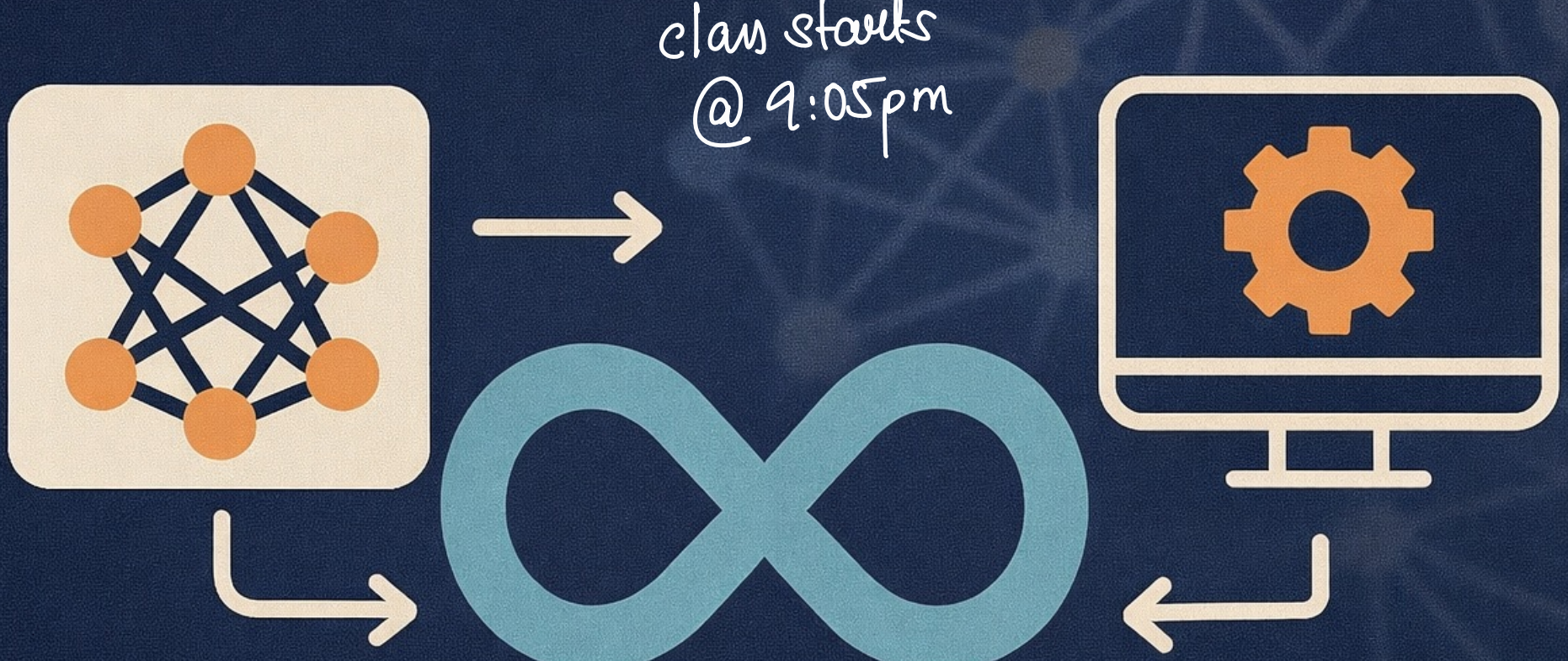
NEURAL NETWORK



Lecture : Neural Network Optimization

#Agenda

1) Weight Initialization
2) Optimizer
3) Activation Functions

#Recap

1) Weight Initialization → Case I: $w = 0$ or $w =$ same value

"No learning"

"Vanishing gradient"

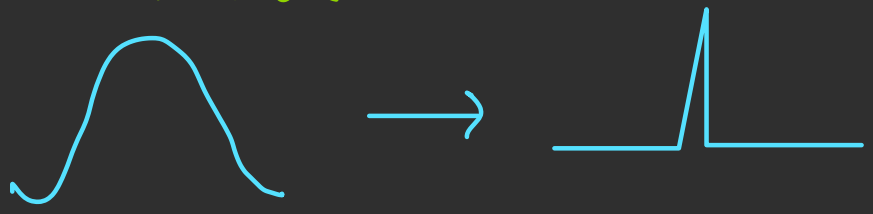
$w_{new} \approx w_{old}$ $\quad \frac{\partial L}{\partial w} \approx 0$

Case II – Random Values → small / large

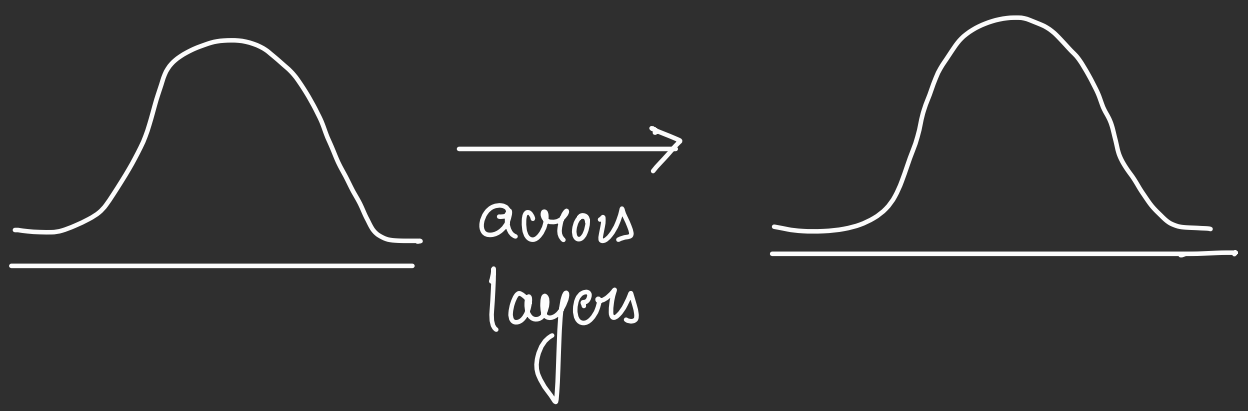
"shrinking distribution problem"

$X \cdot W^T$ ⟶ distribution changes

W → small



1) Xavier / Glorot Initialization



$$Var(X) = \frac{2}{n_{in} + n_{out}}$$

- n_{out} → No of o/p neuron
- n_{in} → No of input Neuron

$$W \sim N\left(0, \frac{2}{n_{in} + n_{out}}\right)$$

When to use?

1) tanh
2) Sigmoid

can produce both neg & pos value

Lay 1

→ output sigmoid

i/p → -ve

2) Kaiming He Initialization

$$W \sim N\left(0, \frac{2}{n_{in}}\right)$$

"He Initialization"

↳ Relu

↳ solves vanishing gradient problem ✓

When to use?

~~✗~~ Relu

↳ $\max(0, z)$

$z = -ve$ $\quad act(z) \Rightarrow 0$

↳ Relu

✓ Half of neurons can get the value

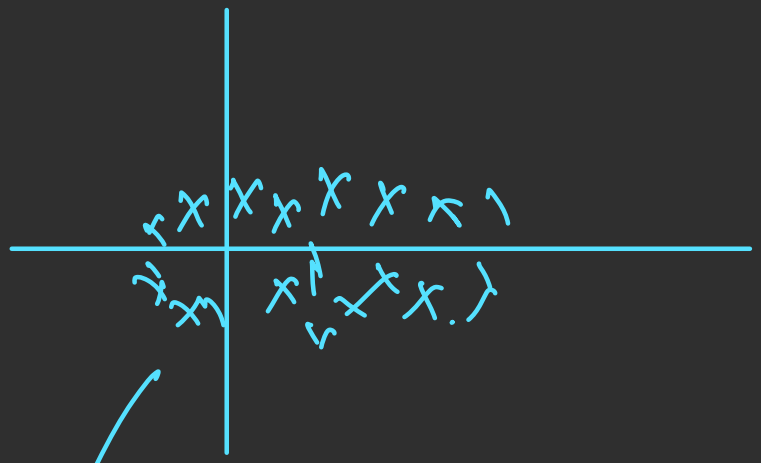
528 → Neuron

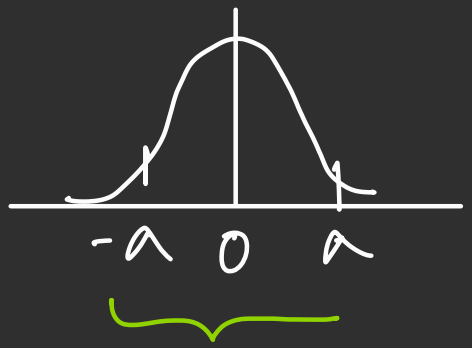
$z_1 = -3.2$ $\quad Relu(z_1) = 0$

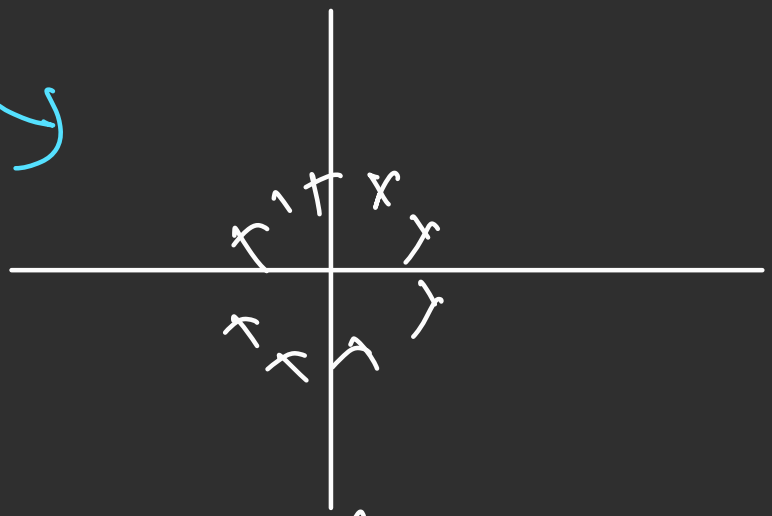
$z_2 = 4.6$ $\quad Relu(z_2) = 4.6$

$z_3 = -3.86$ $\quad Relu(z_3) = 0$









$(\mu = 0, \sigma = 1)$

colA	colB
1	0.63
1	0.82
-1	0.16
1	0.32

Optimizer

Epoch = 20

Data ⇒ 20,000 row

gradient Descent

- Batch GD (20,000 → weight update)
- Schocashtic GD (1 dat pt → weight update)
- Mini_Batch gradient descent

SGD

(1 data pt → weight 1 times

20,000 → 20,000 times)

Ep = 1

Batch size ⇒ 100 data point

↓

1 w update

$\frac{20000}{100}$ ⇒ 200 W update

batch size = 20,000

Batch size = 2^n Format

↳ 2, 4, 8, 16, 32, 64, 128 .....

SGD Vs Batch GD Vs Mini Batch GD

fast but mostly inaccurate

Slower but accurate

faster than BGD but slower than SGD

Accuracy → Better than SGD but less BGD

Speed: SGD >> MGD >> BGD

Accuracy: BGD >> MGD >> SGD

Mini Batch

Dataset = 20,000

BGD → 20000

↳ then update weights

1 time per dataset

Mini Batch → 100 x 200

↓

update weight

200 times per whole dataset

Formula

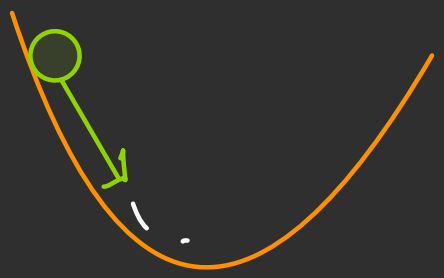
$$W_{new} = W_{old} - \eta \frac{\partial L}{\partial w}$$

step size

learning rate

$0.01 \times 0.01 =$ smaller value

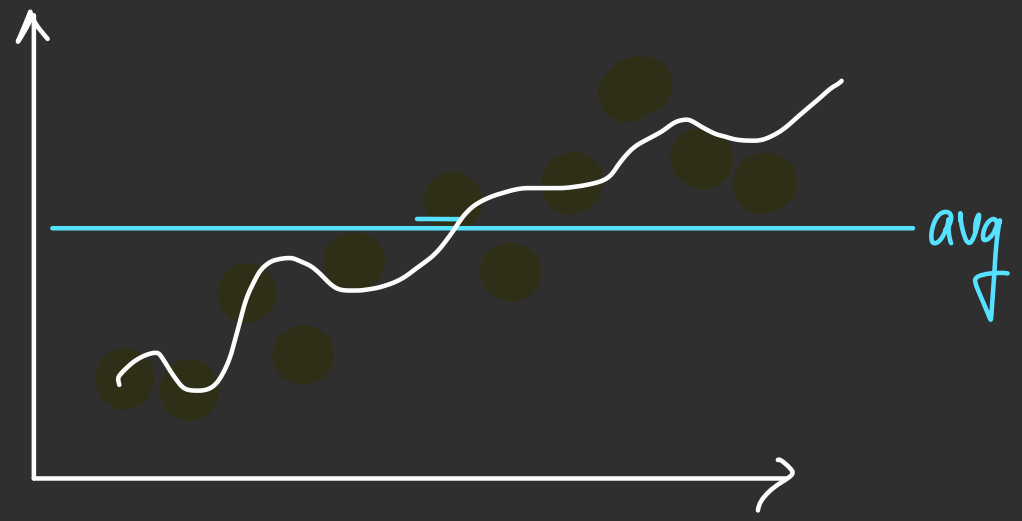
$0.01 \times 1.2 =$ large value



"Non-Convex curves"

New optimization algorithm

1) Exponential weighted moving average.



$$v_t = \beta v_{t-1} + (1-\beta)\Delta_2$$

#idea

Days	Value
D_1	25
D_2	26
D_3	27

$\rightarrow P_1 \rightarrow P_2 \rightarrow P_3 \rightarrow P_4$

EWMA

$$V_t = \beta V_{t-1} + (1-\beta)\Delta t$$

Δt → current gradient step

$V_0 = 25/0$

$V_1 = \beta V_0 + (1-\beta)\Delta t$

$V_2 = \beta V_1 + (1-\beta)\Delta t$

$$V_1 = \beta V_0 + (1-\beta)\Delta w_1$$

EWMA

$$V_2 = \beta V_1 + (1-\beta)\Delta w_2$$

$$\Rightarrow \beta(\beta V_0 + (1-\beta)\Delta w_1) + (1-\beta)\Delta w_2$$

$$\Rightarrow \beta^2 V_0 + \beta(1-\beta)\Delta w_1 + (1-\beta)\Delta w_2$$

$V_0 = 0$

$\beta = 0.9$

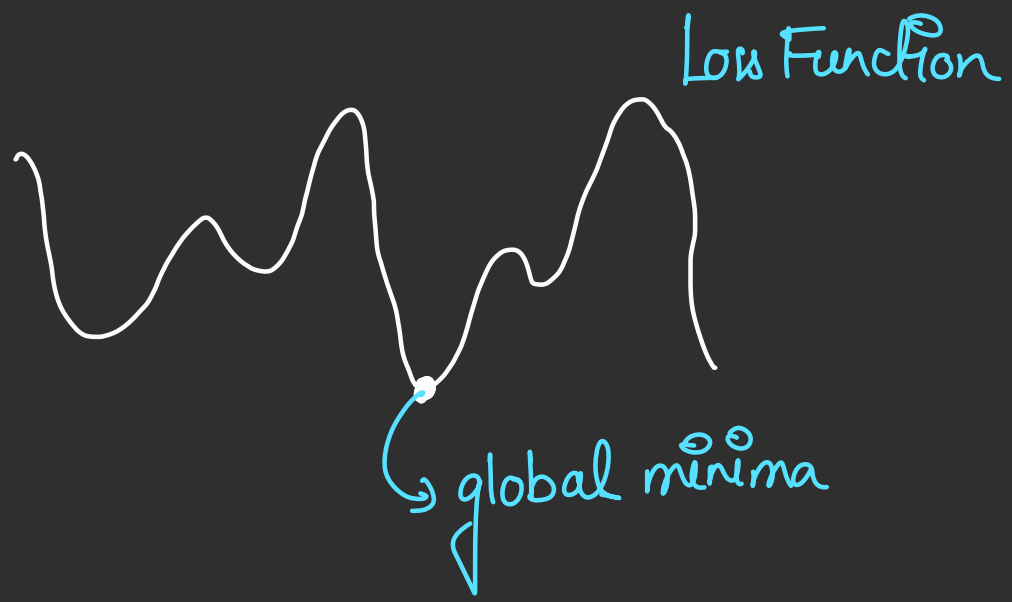
$\Delta w_0 = 0.1$

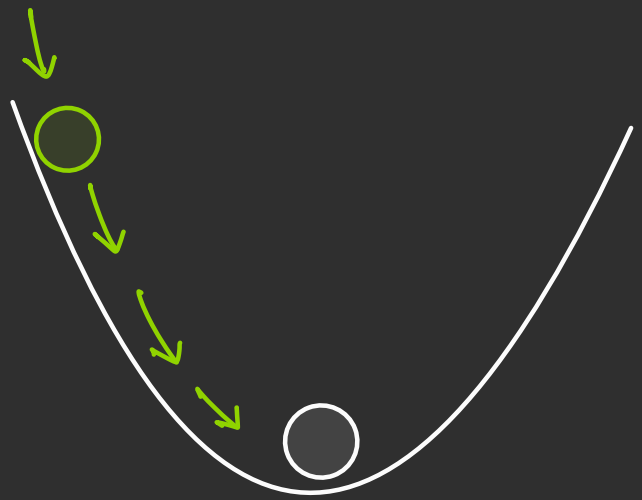
$V_1 = \beta V_0 + (1-\beta)\Delta w_0$

$\Rightarrow (0.9) \times 0 + (0.1) \times 0.1 =$

0.01

Momentum Optimization





#Mathematically

$$W_{t+1} = W_t - \eta \Delta W_t$$

↳ gradient descent

$V \rightarrow$ velocity

$$W_{t+1} = W_t - V_t$$

$$V_t = \beta * V_{t-1} + \eta \Delta W_t$$

↳ $(0 < \beta \leq 1) \simeq 0.9$

(Acts as friction)

$$W_{t+1} = W_t - (\beta V_{t-1} + \eta \Delta W_t)$$

$$W_{new} = W_{old} - \eta \frac{\partial L}{\partial W_{old}}$$

Adagrad (Sparse dataset)

↳ Adaptive gradient

cgpa	iq	iit	Lpa
—	—	1	—
—	—	0	—
—	—	0	—
—	—	0	—

$$\eta \frac{\partial L}{\partial w}$$

Cas I

$$\begin{cases} \frac{\partial L}{\partial w} \uparrow \\ \eta \downarrow \end{cases}$$

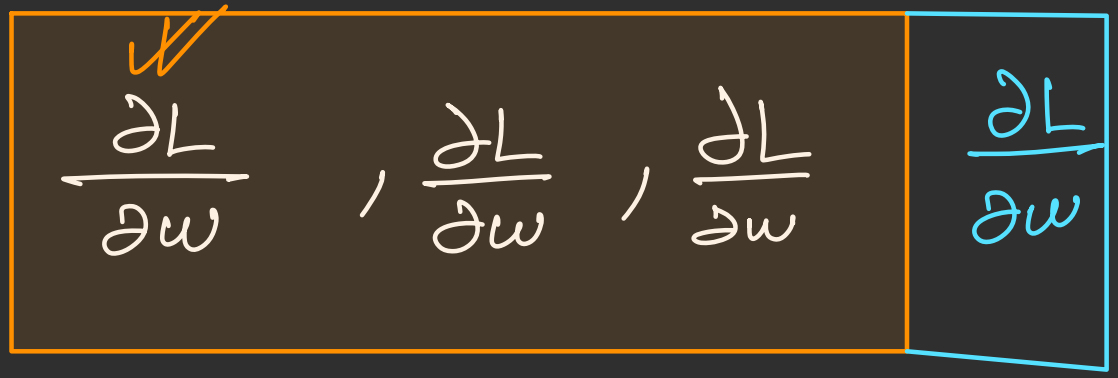
Case II

$$\begin{cases} \frac{\partial L}{\partial w} \downarrow \\ \eta \uparrow \end{cases}$$

RMS prop

↳ Root Mean Square

moving average of that



η $\eta_{(2)}$ $\eta_{(3)}$

Adam

Momentum + RMS prop

→ No local minima problem

→ varying learning rate

Doubts

$$z = \underset{m}{W} x \qquad \begin{cases} x = \text{input vector} \\ W = \text{weight matrix} \end{cases}$$

$$\text{Var}(z) \underset{\sim}{=} \text{Var}(x)$$

Variance of the output

$$z = w_1 x_1 + w_2 x_2 + w_3 x_3 \cdots + w_{n_{in}} x_{n_{in}} + b'$$

$$Var(z) = n_{in} \cdot Var(w) \cdot Var(x)$$

$n_{in} \rightarrow 1$

$$Var(z) = Var(z)$$

$$Var(w) \cdot n_{in} = 1$$

$$\Rightarrow \left(Var(w) = \frac{1}{n_{in}} \right)$$

$$a = \text{Relu}(z)$$

(Relu zeroes out almost half the values)

$$\text{Var}(a) \approx \frac{1}{2}\text{Var}(z)$$

n_{out}

forward variance

n_{out} = weighted sum of input

5 inputs $\longrightarrow$ 1 value

$$z = w_1x_1 + w_2x_2 \cdots w_5x_5 \Rightarrow \boxed{1}$$

$$Var(z) = \frac{1}{2} Var(x) + \frac{n_{out}}{\text{small}}$$

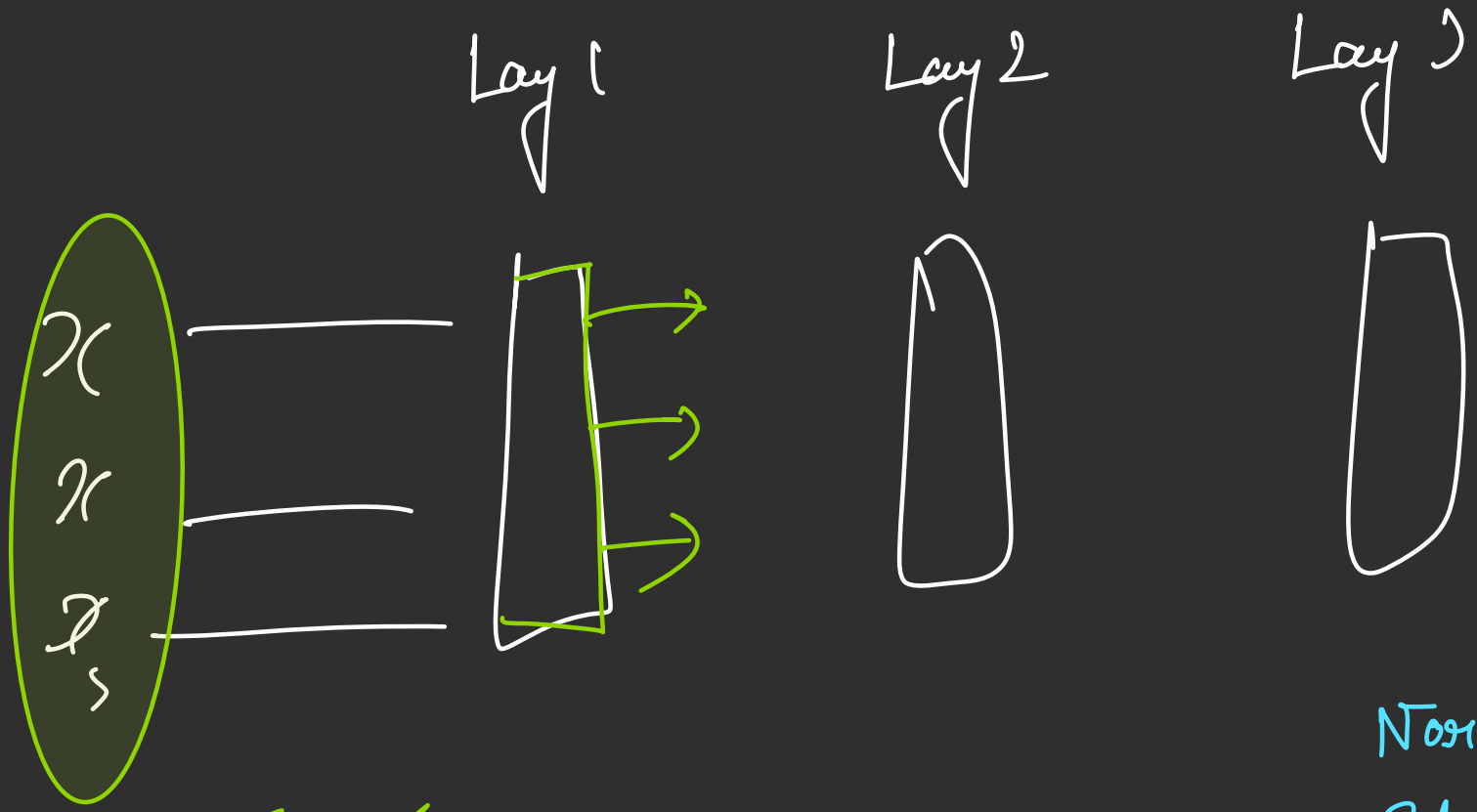
$\Rightarrow$

100K



$\downarrow$ 50%

$-ve \rightarrow 0$



Normaliz -> Max -> (0, 1)

Std -> (-3, +3)

w x

-ve -ve

-ve

$z = -ve \Rightarrow 0$

w x => +ve

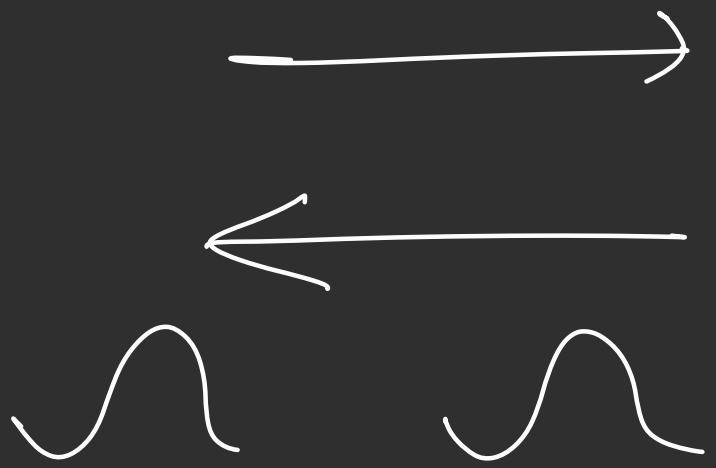
-ve -ve

Xavier

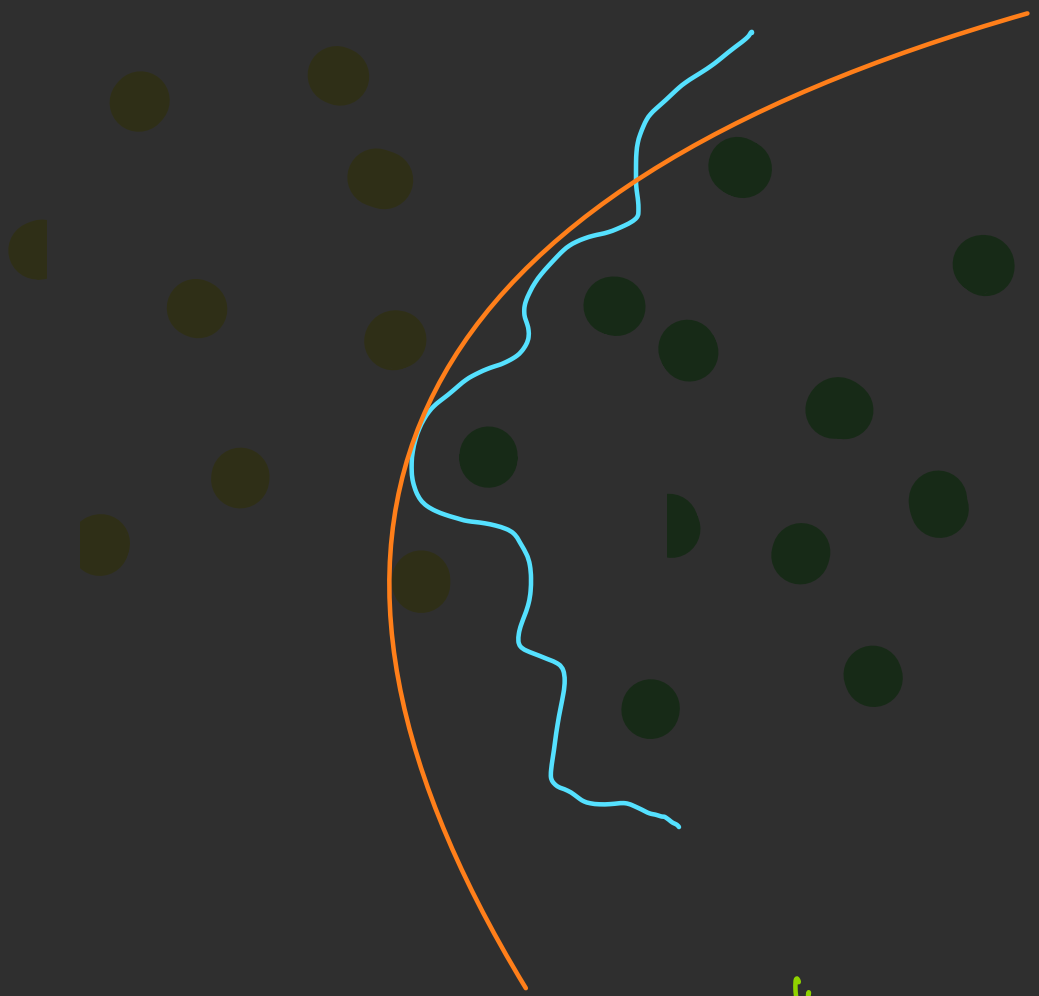
Forward propagative Variance

Backward propagative variance

$$\frac{2}{n_{in} + n_{out}}$$



Relu



I/P	Out
-3	0
-0.5	0
2	2

Introduce Non-linearity

"Dying Relu"